

\$\$\$ sql.banco

DROP TABLE IF EXISTS pecas;

DROP TABLE IF EXISTS sinistros;

DROP TABLE IF EXISTS seguradoras;

```
CREATE TABLE seguradoras (  
    nome VARCHAR(50) PRIMARY KEY,  
    cidade VARCHAR(50),  
    cobertura_percentual INT,  
    possui_atendimento_24h BOOLEAN,  
    forma_pagamento_preferencial VARCHAR(20)  
);
```

```
CREATE TABLE sinistros (  
    segurado VARCHAR(80) PRIMARY KEY,  
    telefone VARCHAR(20),  
    cidade VARCHAR(50),  
    grau_monta VARCHAR(20),  
    perda_total BOOLEAN  
);
```

```
CREATE TABLE pecas (  
    codigo INT PRIMARY KEY,  
    sinistro_segurado VARCHAR(80),  
    nome VARCHAR(50),  
    marca VARCHAR(20),  
    preco DECIMAL(10,2),  
    mao_obra_propria BOOLEAN,  
    tipo_registro VARCHAR(20),  
    tipo_peca_carro VARCHAR(20),  
    tipo_peca_moto VARCHAR(20),  
    dias_garantia INT,  
    cor VARCHAR(20),  
    FOREIGN KEY (sinistro_segurado) REFERENCES sinistros(segurado)  
);
```

\$\$\$ src.controles.ControladorCadastroPecas

```
package controles;
```

```
import entidades.Pecas;
```

```
import interfaces.JanelaCadastroPecas;
```

```
import java.awt.Frame;
```

```
public class ControladorCadastroPecas {
```

```
    public ControladorCadastroPecas() {  
        this(null);  
    }
```

```
    public ControladorCadastroPecas(Frame owner) {  
        JanelaCadastroPecas janela = new JanelaCadastroPecas(this, owner);  
        janela.setVisible(true);  
        janelaToFront();  
        janela.requestFocus();  
    }
```

```
    public String inserirPecas(Pecas pecas) {  
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getCodigo());  
        if (peca_buscada == null) return Pecas.inserirPecas(pecas);  
        else return "Código de Peça já cadastrado";  
    }
```

```
    public String alterarPecas(Pecas pecas) {  
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getCodigo());  
        if (peca_buscada != null) return Pecas.alterarPecas(pecas);  
        else return "Código de Peça não cadastrado";  
    }
```

```
    public String removerPecas(int codigo) {  
        Pecas peca_buscada = Pecas.buscarPecas(codigo);  
        if (peca_buscada != null) return Pecas.removerPecas(codigo);  
        else return "Código de Peça não cadastrado";  
    }
```

}

}

\$\$\$ src.controles.ControladorCadastroSeguradoras

```
package controles;

import entidades.Seguradora;
import interfaces.JanelaCadastroSeguradoras;
import java.awt.Frame;

public class ControladorCadastroSeguradoras {

    public ControladorCadastroSeguradoras() {
        this(null);
    }

    public ControladorCadastroSeguradoras(Frame owner) {
        JanelaCadastroSeguradoras janela = new JanelaCadastroSeguradoras(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada == null) return Seguradora.inserirSeguradora(seguradora);
        else return "Nome de Seguradora já cadastrado";
    }

    public String alterarSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada != null) return Seguradora.alterarSeguradora(seguradora);
        else return "Nome de Seguradora não cadastrado";
    }

    public String removerSeguradora(String nome) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(nome);
        if (seguradora_buscada != null) return Seguradora.removerSeguradora(nome);
        else return "Nome de Seguradora não cadastrado";
    }
}
```

}

}

\$\$\$ src.controles.ControladorCadastroSinistros

```
package controles;

import entidades.Sinistro;
import interfaces.JanelaCadastroSinistros;

public class ControladorCadastroSinistros {

    public ControladorCadastroSinistros() {
        new JanelaCadastroSinistros(this).setVisible(true);
    }

    public String inserirSinistro(Sinistro sinistro) {
        Sinistro sinistro_buscado = Sinistro.buscarSinistro(sinistro.getSegurado());
        if (sinistro_buscado == null) return Sinistro.inserirSinistro(sinistro);
        else return "Segurado de Sinistro já cadastrado";
    }

    public String alterarSinistro(Sinistro sinistro) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(sinistro.getSegurado());
        if (sinistro1 != null) return Sinistro.alterarSinistro(sinistro);
        else return "Segurado de Sinistro não cadastrado";
    }

    public String removerSinistro(String segurado) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(segurado);
        if (sinistro1 != null) return Sinistro.removerSinistro(segurado);
        else return "Segurado de Sinistro não cadastrado";
    }
}
```

\$\$\$ src.entidades.Pecas

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Pecas {

    public enum MarcaPeca {
        OEM("oem"),
        ORIGINAL("original"),
        GENUINA("genuina");

        private final String texto;

        MarcaPeca(String texto) {
            this.texto = texto;
        }

        public static MarcaPeca fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Marca da peça não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT)
                .replace("á", "a")
                .replace("à", "a")
                .replace("â", "a")
                .replace("ã", "a")
                .replace("é", "e")
                .replace("ê", "e")
                .replace("í", "i")
```

```
.replace("ó", "o")
.replace("ô", "o")
.replace("õ", "o")
.replace("ú", "u")
.replace("ç", "c");
```

```
if (normalizado.equals("oem")) return OEM;
if (normalizado.equals("original")) return ORIGINAL;
if (normalizado.equals("genuina")) return GENUINA;
```

```
throw new IllegalArgumentException("Marca da peça inválida: " + texto);
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
public enum TipoPeca {
    FAROL("farol"),
    PARACHOQUES("parachoques"),
    CAPO("capo"),
    LANTERNAS("lanternas"),
    PORTAS("portas"),
    ESCAPAMENTO("escapamento"),
    PISTAO("pistao"),
    BIELA("biela"),
    CABECOTE("cabecote");
```

```
private final String texto;
```

```
TipoPeca(String texto) {
    this.texto = texto;
}
```

```
public static TipoPeca fromTexto(String texto) {
```



```
if (texto == null) {  
    throw new IllegalArgumentException("Tipo da peça não informado");  
}
```

```
String normalizado = texto.trim().toLowerCase(Locale.ROOT)
```

```
    .replace("á", "a")  
    .replace("à", "a")  
    .replace("â", "a")  
    .replace("ã", "a")  
    .replace("é", "e")  
    .replace("ê", "e")  
    .replace("í", "i")  
    .replace("ó", "o")  
    .replace("ô", "o")  
    .replace("õ", "o")  
    .replace("ú", "u")  
    .replace("ç", "c");
```

```
if (normalizado.equals("farol")) return FAROL;  
if (normalizado.equals("parachoques")) return PARACHOQUES;  
if (normalizado.equals("capo")) return CAPO;  
if (normalizado.equals("lanternas")) return LANTERNAS;  
if (normalizado.equals("portas")) return PORTAS;  
if (normalizado.equals("escapamento")) return ESCAPAMENTO;  
if (normalizado.equals("pistao")) return PISTAO;  
if (normalizado.equals("biela")) return BIELA;  
if (normalizado.equals("cabecote")) return CABECOTE;
```

```
    throw new IllegalArgumentException("Tipo da peça inválido: " + texto);  
}
```

```
@Override
```

```
public String toString() {  
    return texto;  
}  
}
```

```

public enum TipoRegistro {
    GERAL("geral"),
    MECANICA("mecanica"),
    LATARIA("lataria");

    private final String texto;

    TipoRegistro(String texto) {
        this.texto = texto;
    }

    public static TipoRegistro fromTexto(String texto) {
        if (texto == null || texto.trim().isEmpty()) {
            return GERAL;
        }

        String normalizado = texto.trim().toLowerCase(Locale.ROOT);
        if (normalizado.equals("mecanica") || normalizado.equals("mecânica")) return MECANICA;
        if (normalizado.equals("lataria")) return LATARIA;
        return GERAL;
    }

    @Override
    public String toString() {
        return texto;
    }
}

private int codigo;
private String sinistro_segurado;
private String nome;
private MarcaPeca marca;
private double preco;
private boolean mao_obra_propria;
private TipoPeca tipo;
private Integer dias_garantia;
private String cor;

```

```
private TipoRegistro tipo_registro;
```

```
public Pecas() {  
    this(0, null, null, null, 0.0, false, null, null, null, TipoRegistro.GERAL);  
}
```

```
public Pecas(  
    int codigo,  
    String nome,  
    MarcaPeca marca,  
    double preco,  
    boolean mao_obra_propria  
) {  
    this(codigo, null, nome, marca, preco, mao_obra_propria, null, null, null,  
TipoRegistro.GERAL);  
}
```

```
public Pecas(  
    int codigo,  
    String nome,  
    MarcaPeca marca,  
    double preco,  
    TipoPeca tipo,  
    String cor,  
    boolean mao_obra_propria  
) {  
    this(codigo, null, nome, marca, preco, mao_obra_propria, tipo, null, cor,  
inferirTipoRegistro(tipo));  
}
```

```
public Pecas(  
    int codigo,  
    String nome,  
    MarcaPeca marca,  
    double preco,  
    TipoPeca tipo,  
    String cor,
```

```

        Integer dias_garantia,
        boolean mao_obra_propria
    ) {
        this(codigo, null, nome, marca, preco, mao_obra_propria, tipo, dias_garantia, cor,
inferirTipoRegistro(tipo));
    }

```

```

public Pecas(
    int codigo,
    String sinistro_segurado,
    String nome,
    MarcaPeca marca,
    double preco,
    boolean mao_obra_propria,
    TipoPeca tipo,
    Integer dias_garantia,
    String cor,
    TipoRegistro tipo_registro
) {
    this.codigo = codigo;
    this.sinistro_segurado = sinistro_segurado;
    this.nome = nome;
    this.marca = marca;
    this.preco = preco;
    this.mao_obra_propria = mao_obra_propria;
    this.tipo = tipo;
    this.dias_garantia = dias_garantia;
    this.cor = cor;
    this.tipo_registro = tipo_registro != null ? tipo_registro : inferirTipoRegistro(tipo);
}

```

```

private static TipoRegistro inferirTipoRegistro(TipoPeca tipo) {
    if (tipo == null) {
        return TipoRegistro.GERAL;
    }

```

```

    switch (tipo) {

```

```
        case FAROL:
        case PARACHOQUES:
        case CAPO:
        case LANTERNAS:
        case PORTAS:
            return TipoRegistro.LATARIA;
        default:
            return TipoRegistro.MECANICA;
    }
}
```

```
public int getCodigo() {
    return codigo;
}
```

```
public String getSinistroSegurado() {
    return sinistro_segurado;
}
```

```
public String getNome() {
    return nome;
}
```

```
public MarcaPeca getMarca() {
    return marca;
}
```

```
public MarcaPeca getCategoria() {
    return marca;
}
```

```
public double getPreco() {
    return preco;
}
```

```
public boolean getMaoObraPropria() {
    return mao_obra_propria;
}
```

```
}
```

```
public boolean isMaoObraPropria() {  
    return mao_obra_propria;  
}
```

```
public boolean getMaoDeObra() {  
    return mao_obra_propria;  
}
```

```
public TipoPeca getTipo() {  
    return tipo;  
}
```

```
public Integer getDiasGarantia() {  
    return dias_garantia;  
}
```

```
public String getCor() {  
    return cor;  
}
```

```
public TipoRegistro getTipoRegistro() {  
    return tipo_registro;  
}
```

```
public void setCodigo(int codigo) {  
    this.codigo = codigo;  
}
```

```
public void setSinistroSegurado(String sinistro_segurado) {  
    this.sinistro_segurado = sinistro_segurado;  
}
```

```
public void setNome(String nome) {  
    this.nome = nome;  
}
```

```
public void setMarca(MarcaPeca marca) {  
    this.marca = marca;  
}
```

```
public void setCategoria(MarcaPeca categoria) {  
    this.marca = categoria;  
}
```

```
public void setPreco(double preco) {  
    this.preco = preco;  
}
```

```
public void setMaoObraPropria(boolean mao_obra_propria) {  
    this.mao_obra_propria = mao_obra_propria;  
}
```

```
public void setMaoDeObra(boolean mao_obra_propria) {  
    this.mao_obra_propria = mao_obra_propria;  
}
```

```
public void setTipo(TipoPeca tipo) {  
    this.tipo = tipo;  
    if (this.tipo_registro == TipoRegistro.GERAL) {  
        this.tipo_registro = inferirTipoRegistro(tipo);  
    }  
}
```

```
public void setDiasGarantia(Integer dias_garantia) {  
    this.dias_garantia = dias_garantia;  
}
```

```
public void setCor(String cor) {  
    this.cor = cor;  
}
```

```
public void setTipoRegistro(TipoRegistro tipo_registro) {
```

```
        this.tipo_registro = tipo_registro != null ? tipo_registro : TipoRegistro.GERAL;
    }
}
```

```
public Pecas getVisao() {
    return new Pecas(
        codigo,
        sinistro_segurado,
        nome,
        marca,
        preco,
        mao_obra_propria,
        tipo,
        dias_garantia,
        cor,
        tipo_registro
    );
}
```

```
@Override
public String toString() {
    return codigo + " - " + nome + " [" + marca + "]";
}
```

```
private static Pecas criarVisao(ResultSet resultado) throws SQLException {
    int codigo = resultado.getInt("codigo");
    String sinistro_segurado = resultado.getString("sinistro_segurado");
    String nome = resultado.getString("nome");
    MarcaPeca marca = MarcaPeca.fromTexto(resultado.getString("marca"));
    double preco = resultado.getDouble("preco");
    boolean mao_obra_propria = resultado.getBoolean("mao_obra_propria");
    TipoRegistro tipo_registro = TipoRegistro.fromTexto(resultado.getString("tipo_registro"));
    TipoPeca tipo = null;
    String tipoTexto = null;
    String cor = resultado.getString("cor");
    Integer dias_garantia = null;

    if (tipo_registro == TipoRegistro.MECANICA) {
```



```

        tipoTexto = resultado.getString("tipo_peca_moto");
        dias_garantia = resultado.getObject("dias_garantia") != null
            ? resultado.getInt("dias_garantia")
            : null;
    } else if (tipo_registro == TipoRegistro.LATARIA) {
        tipoTexto = resultado.getString("tipo_peca_carro");
    }

    if (tipoTexto != null && !tipoTexto.trim().isEmpty()) {
        tipo = TipoPeca.fromTexto(tipoTexto);
    }

    if (tipo_registro == TipoRegistro.GERAL && resultado.getObject("tipo_peca_carro") != null)
    {
        tipoTexto = resultado.getString("tipo_peca_carro");
        if (tipoTexto != null && !tipoTexto.trim().isEmpty()) {
            tipo = TipoPeca.fromTexto(tipoTexto);
        }
    }

    if (tipo_registro == TipoRegistro.GERAL && resultado.getObject("tipo_peca_moto") != null)
    {
        tipoTexto = resultado.getString("tipo_peca_moto");
        if (tipoTexto != null && !tipoTexto.trim().isEmpty()) {
            tipo = TipoPeca.fromTexto(tipoTexto);
        }
        if (resultado.getObject("dias_garantia") != null) {
            dias_garantia = resultado.getInt("dias_garantia");
        }
    }

    return new Pecas(
        codigo,
        sinistro_segurado,
        nome,
        marca,
        preco,

```

```

        mao_obra_propria,
        tipo,
        dias_garantia,
        cor,
        tipo_registro
    );
}

```

```

private static Pecas criarVisaoLegado(ResultSet resultado) throws SQLException {
    int codigo = resultado.getInt("codigo");
    String sinistro_segurado = resultado.getString("sinistro_numero");
    String nome = resultado.getString("nome");
    MarcaPeca marca = MarcaPeca.fromTexto(resultado.getString("categoria"));
    double preco = resultado.getDouble("preco");
    boolean mao_obra_propria = resultado.getBoolean("mao_obra_propria");
    String tipoTexto = resultado.getString("tipo");
    String cor = resultado.getString("cor");
    TipoPeca tipo = null;

    if (tipoTexto != null && !tipoTexto.trim().isEmpty()) {
        try {
            tipo = TipoPeca.fromTexto(tipoTexto);
        } catch (IllegalArgumentException excecao_enum) {
            tipo = null;
        }
    }

    return new Pecas(
        codigo,
        sinistro_segurado,
        nome,
        marca,
        preco,
        mao_obra_propria,
        tipo,
        null,
        cor,

```

```
        inferirTipoRegistro(tipo)
    );
}
```

```
public static Pecas[] getVisoes() {
    String sql = "SELECT codigo, sinistro_segurado, nome, marca, preco, mao_obra_propria,
tipo_registro, tipo_peca_carro, tipo_peca_moto, dias_garantia, cor FROM pecas";
    ResultSet lista_resultados = null;
    ArrayList<Pecas> visoes = new ArrayList<>();

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {
                visoes.add(criarVisao(lista_resultados));
            } catch (IllegalArgumentException excecao_enum) {
                // Ignora registros com valor fora do domínio esperado.
            }
        }

        lista_resultados.close();
        comando.close();
    } catch (SQLException excecao_sql) {
        try {
            String sql_legacy = "SELECT codigo, sinistro_numero, nome, categoria, preco, tipo, cor,
mao_obra_propria FROM pecas";
            PreparedStatement comando_legacy = BD.conexao.prepareStatement(sql_legacy);
            lista_resultados = comando_legacy.executeQuery();

            while (lista_resultados.next()) {
                try {
                    visoes.add(criarVisaoLegacy(lista_resultados));
                } catch (IllegalArgumentException excecao_enum) {
                    // Ignora registros com valor fora do domínio esperado.
                }
            }
        }
    }
}
```

```

    }

    lista_resultados.close();
    comando_legacy.close();
} catch (SQLException excecao_legacy) {
    excecao_legacy.printStackTrace();
}
}

return visoes.toArray(new Pecas[visoes.size()]);
}

```

```

public static Pecas buscarPecas(String nome) {
    String sql = "SELECT codigo, sinistro_segurado, nome, marca, preco, mao_obra_propria,
tipo_registro, tipo_pecas_carro, tipo_pecas_moto, dias_garantia, cor FROM pecas WHERE nome
= ?";

    ResultSet lista_resultados = null;
    Pecas pecas = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {
                pecas = criarVisao(lista_resultados);
            } catch (IllegalArgumentException excecao_enum) {
                pecas = null;
            }
        }

        lista_resultados.close();
        comando.close();
    } catch (SQLException excecao_sql) {
        try {
            String sql_legacy = "SELECT codigo, sinistro_numero, nome, categoria, preco, tipo, cor,
mao_obra_propria FROM pecas WHERE nome = ?";

```

```

PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
comando_legado.setString(1, nome);
lista_resultados = comando_legado.executeQuery();

while (lista_resultados.next()) {
    try {
        pecas = criarVisaoLegado(lista_resultados);
    } catch (IllegalArgumentException excecao_enum) {
        pecas = null;
    }
}

lista_resultados.close();
comando_legado.close();
} catch (SQLException excecao_legado) {
    excecao_legado.printStackTrace();
    pecas = null;
}
}

return pecas;
}

```

```

public static Pecas buscarPecas(int codigo) {
    String sql = "SELECT codigo, sinistro_segurado, nome, marca, preco, mao_obra_propria,
tipo_registro, tipo_pecas_carro, tipo_pecas_moto, dias_garantia, cor FROM pecas WHERE codigo
= ?";

    ResultSet lista_resultados = null;
    Pecas pecas = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, codigo);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {

```

```

        pecas = criarVisao(lista_resultados);
    } catch (IllegalArgumentException excecao_enum) {
        pecas = null;
    }
}

lista_resultados.close();
comando.close();
} catch (SQLException excecao_sql) {
    try {
        String sql_legado = "SELECT codigo, sinistro_numero, nome, categoria, preco, tipo, cor,
maoobra_propria FROM pecas WHERE codigo = ?";
        PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
        comando_legado.setInt(1, codigo);
        lista_resultados = comando_legado.executeQuery();

        while (lista_resultados.next()) {
            try {
                pecas = criarVisaoLegado(lista_resultados);
            } catch (IllegalArgumentException excecao_enum) {
                pecas = null;
            }
        }

        lista_resultados.close();
        comando_legado.close();
    } catch (SQLException excecao_legado) {
        excecao_legado.printStackTrace();
        pecas = null;
    }
}

return pecas;
}

```

```

public static Pecas[] buscarPecasPorSinistro(String sinistro_segurado) {
    String sql = "SELECT codigo, sinistro_segurado, nome, marca, preco, maoobra_propria,

```

```
tipo_registro, tipo_peca_carro, tipo_peca_moto, dias_garantia, cor FROM pecas WHERE sinistro_segurado = ?";
```

```
ResultSet lista_resultados = null;
```

```
ArrayList<Pecas> visoes = new ArrayList<>();
```

```
try {
```

```
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
```

```
    comando.setString(1, sinistro_segurado);
```

```
    lista_resultados = comando.executeQuery();
```

```
    while (lista_resultados.next()) {
```

```
        try {
```

```
            visoes.add(criarVisao(lista_resultados));
```

```
        } catch (IllegalArgumentException excecao_enum) {
```

```
            // Ignora registros inválidos.
```

```
        }
```

```
    }
```

```
    lista_resultados.close();
```

```
    comando.close();
```

```
} catch (SQLException excecao_sql) {
```

```
    try {
```

```
        String sql_legado = "SELECT codigo, sinistro_numero, nome, categoria, preco, tipo, cor, mao_obra_propria FROM pecas WHERE sinistro_numero = ?";
```

```
        PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
```

```
        comando_legado.setString(1, sinistro_segurado);
```

```
        lista_resultados = comando_legado.executeQuery();
```

```
        while (lista_resultados.next()) {
```

```
            try {
```

```
                visoes.add(criarVisaoLegado(lista_resultados));
```

```
            } catch (IllegalArgumentException excecao_enum) {
```

```
                // Ignora registros inválidos.
```

```
            }
```

```
        }
```

```
        lista_resultados.close();
```

```

        comando_legado.close();
    } catch (SQLException excecao_legado) {
        excecao_legado.printStackTrace();
    }
}

return visoes.toArray(new Pecas[visoes.size()]);
}

```

```

public static String inserirPecas(Pecas pecas) {
    String sql = "INSERT INTO pecas (codigo, sinistro_segurado, nome, marca, preco,
mao_obra_propria, tipo_registro, tipo_pecas_carro, tipo_pecas_moto, dias_garantia, cor) VALUES
(?,?,?,?,?,?,?,?,?,?,?)";

```

```

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, pecas.getCodigo());
        comando.setString(2, pecas.getSinistroSegurado());
        comando.setString(3, pecas.getNome());
        comando.setString(4, pecas.getMarca().toString());
        comando.setDouble(5, pecas.getPreco());
        comando.setBoolean(6, pecas.getMaoObraPropria());
        comando.setString(7, pecas.getTipoRegistro().toString());
        comando.setString(8, pecas.getTipo() != null && pecas.getTipoRegistro() ==
TipoRegistro.LATARIA
            ? pecas.getTipo().toString()
            : null);
        comando.setString(9, pecas.getTipo() != null && pecas.getTipoRegistro() ==
TipoRegistro.MECANICA
            ? pecas.getTipo().toString()
            : null);
        if (pecas.getDiasGarantia() != null) {
            comando.setInt(10, pecas.getDiasGarantia());
        } else {
            comando.setNull(10, java.sql.Types.INTEGER);
        }
        comando.setString(11, pecas.getCor());
        comando.executeUpdate();
    }
}

```



```

        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        return inserirPecasLegado(pecas);
    }
}

```

```

public static String alterarPecas(Pecas pecas) {
    String sql = "UPDATE pecas SET sinistro_segurado = ?, nome = ?, marca = ?, preco = ?,
mao_obra_propria = ?, tipo_registro = ?, tipo_peca_carro = ?, tipo_peca_moto = ?, dias_garantia
= ?, cor = ? WHERE codigo = ?";

```

```

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, pecas.getSinistroSegurado());
        comando.setString(2, pecas.getNome());
        comando.setString(3, pecas.getMarca().toString());
        comando.setDouble(4, pecas.getPreco());
        comando.setBoolean(5, pecas.getMaoObraPropria());
        comando.setString(6, pecas.getTipoRegistro().toString());
        comando.setString(7, pecas.getTipo() != null && pecas.getTipoRegistro() ==
TipoRegistro.LATARIA
            ? pecas.getTipo().toString()
            : null);
        comando.setString(8, pecas.getTipo() != null && pecas.getTipoRegistro() ==
TipoRegistro.MECANICA
            ? pecas.getTipo().toString()
            : null);
        if (pecas.getDiasGarantia() != null) {
            comando.setInt(9, pecas.getDiasGarantia());
        } else {
            comando.setNull(9, java.sql.Types.INTEGER);
        }
        comando.setString(10, pecas.getCor());
        comando.setInt(11, pecas.getCodigo());
        comando.executeUpdate();
        comando.close();
        return null;
    }
}

```

```

    } catch (SQLException excecao_sql) {
        return alterarPecasLegado(pecas);
    }
}

```

```

public static String removerPecas(String nome) {
    String sql = "DELETE FROM pecas WHERE nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        return removerPecasLegado(nome);
    }
}

```

```

public static String removerPecas(int codigo) {
    String sql = "DELETE FROM pecas WHERE codigo = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, codigo);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        return removerPecasLegado(codigo);
    }
}

```

```

private static int ultimoCodigoLegado() {
    String sql = "SELECT MAX(codigo) FROM pecas";
    ResultSet lista_resultados = null;
    int codigo = 0;
}

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    lista_resultados = comando.executeQuery();

    while (lista_resultados.next()) {
        codigo = lista_resultados.getInt(1);
    }

    lista_resultados.close();
    comando.close();
} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return codigo;
}

```

```

private static String inserirPecasLegado(Pecas pecas) {
    String sql = "INSERT INTO pecas (codigo, sinistro_numero, nome, categoria, preco, tipo, cor,
mao_obra_propria) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    comando.setInt(1, pecas.getCodigo());
    if (pecas.getSinistroSegurado() != null) {
        try {
            comando.setInt(2, Integer.parseInt(pecas.getSinistroSegurado()));
        } catch (NumberFormatException excecao_numero) {
            comando.setNull(2, java.sql.Types.INTEGER);
        }
    } else {
        comando.setNull(2, java.sql.Types.INTEGER);
    }

    comando.setString(3, pecas.getNome());
    comando.setString(4, pecas.getMarca().toString());
    comando.setDouble(5, pecas.getPreco());

```

```

        comando.setString(6, pecas.getTipo() != null ? pecas.getTipo().toString() : null);
        comando.setString(7, pecas.getCor());
        comando.setBoolean(8, pecas.getMaoObraPropria());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Inserção da Peça no BD";
    }
}

```

```

private static String alterarPecasLegado(Pecas pecas) {
    String sql = "UPDATE pecas SET sinistro_numero = ?, nome = ?, categoria = ?, preco = ?,
tipo = ?, cor = ?, mao_obra_propria = ? WHERE codigo = ?";

```

```

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        if (pecas.getSinistroSegurado() != null) {
            try {
                comando.setInt(1, Integer.parseInt(pecas.getSinistroSegurado()));
            } catch (NumberFormatException excecao_numero) {
                comando.setNull(1, java.sql.Types.INTEGER);
            }
        } else {
            comando.setNull(1, java.sql.Types.INTEGER);
        }
        comando.setString(2, pecas.getNome());
        comando.setString(3, pecas.getMarca().toString());
        comando.setDouble(4, pecas.getPreco());
        comando.setString(5, pecas.getTipo() != null ? pecas.getTipo().toString() : null);
        comando.setString(6, pecas.getCor());
        comando.setBoolean(7, pecas.getMaoObraPropria());
        comando.setInt(8, pecas.getCodigo());
        comando.executeUpdate();
        comando.close();
        return null;
    }
}

```

```

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração da Peça no BD";
    }
}

```

```

private static String removerPecasLegado(String nome) {
    String sql = "DELETE FROM pecas WHERE nome = ?";
    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção da Peça no BD";
    }
}

```

```

private static String removerPecasLegado(int codigo) {
    String sql = "DELETE FROM pecas WHERE codigo = ?";
    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, codigo);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção da Peça no BD";
    }
}
}

```

\$\$\$ src.entidades.Seguradora

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Seguradora {

    public enum FormaPagamentoPreferencial {
        BOLETO("boleto"),
        CARTAO("cartao"),
        PIX("pix"),
        DEBITO_AUTOMATICO("debito_automatico");

        private final String texto;

        FormaPagamentoPreferencial(String texto) {
            this.texto = texto;
        }

        public static FormaPagamentoPreferencial fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Forma de pagamento não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT);

            if (normalizado.equals("boleto")) return BOLETO;
            if (normalizado.equals("cartao") || normalizado.equals("cartão")) return CARTAO;
            if (normalizado.equals("pix")) return PIX;
            if (normalizado.equals("debito_automatico")
                || normalizado.equals("debito automatico"))
```

```
        || normalized.equals("débito_automático")
        || normalized.equals("débito automático")) {
    return DEBITO_AUTOMATICO;
}
```

```
    throw new IllegalArgumentException("Forma de pagamento inválida: " + texto);
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
private String nome;
private String cidade;
private double cobertura_percentual;
private boolean possui_atendimento_24h;
private FormaPagamentoPreferencial forma_pagamento_preferencial;
```

```
public String getNome() { return nome; }
public String getCidade() { return cidade; }
public double getCoberturaPercentual() { return cobertura_percentual; }
public boolean getPossuiAtendimento24h() { return possui_atendimento_24h; }
public boolean isPossuiAtendimento24h() { return possui_atendimento_24h; }
public FormaPagamentoPreferencial getFormaPagamentoPreferencial() { return
forma_pagamento_preferencial; }
```

```
public void setCidade(String cidade) { this.cidade = cidade; }
public void setCoberturaPercentual(double cobertura_percentual) { this.cobertura_percentual =
cobertura_percentual; }
public void setPossuiAtendimento24h(boolean possui_atendimento_24h)
{ this.possui_atendimento_24h = possui_atendimento_24h; }
public void setFormaPagamentoPreferencial(FormaPagamentoPreferencial
forma_pagamento_preferencial) {
    this.forma_pagamento_preferencial = forma_pagamento_preferencial;
}
```

```

public Seguradora(
    String nome,
    String cidade,
    double cobertura_percentual,
    boolean possui_atendimento_24h,
    FormaPagamentoPreferencial forma_pagamento_preferencial
) {
    this.nome = nome;
    this.cidade = cidade;
    this.cobertura_percentual = cobertura_percentual;
    this.possui_atendimento_24h = possui_atendimento_24h;
    this.forma_pagamento_preferencial = forma_pagamento_preferencial;
}

public Seguradora(String nome, String cidade, double cobertura_percentual) {
    this(nome, cidade, cobertura_percentual, false, FormaPagamentoPreferencial.BOLETO);
}

public Seguradora(String nome, String cidade) {
    this(nome, cidade, 0, false, FormaPagamentoPreferencial.BOLETO);
}

public String toString() {
    return nome + " [" + cidade + "]";
}

public Seguradora getVisao() {
    return new Seguradora(
        nome,
        cidade,
        cobertura_percentual,
        possui_atendimento_24h,
        forma_pagamento_preferencial
    );
}

public static Seguradora[] getVisoes() {

```



```

String sql = "SELECT nome, cidade, cobertura_percentual, possui_atendimento_24h,
forma_pagamento_preferencial FROM seguradoras";

ResultSet lista_resultados = null;
ArrayList<Seguradora> visoes = new ArrayList<>();

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    lista_resultados = comando.executeQuery();

    while (lista_resultados.next()) {
        String nome = lista_resultados.getString("nome");
        String cidade = lista_resultados.getString("cidade");
        double cobertura_percentual = lista_resultados.getDouble("cobertura_percentual");
        boolean possui_atendimento_24h =
lista_resultados.getBoolean("possui_atendimento_24h");
        FormaPagamentoPreferencial forma_pagamento_preferencial =
            FormaPagamentoPreferencial.fromTexto(
                lista_resultados.getString("forma_pagamento_preferencial")
            );
        visoes.add(new Seguradora(
            nome,
            cidade,
            cobertura_percentual,
            possui_atendimento_24h,
            forma_pagamento_preferencial
        ));
    }

    lista_resultados.close();
    comando.close();

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return visoes.toArray(new Seguradora[visoes.size()]);
}

```

```

public static Seguradora buscarSeguradora(String nome) {
    String sql = "SELECT * FROM seguradoras WHERE nome = ?";
    ResultSet lista_resultados = null;
    Seguradora seguradora = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            seguradora = new Seguradora(
                nome,
                lista_resultados.getString("cidade"),
                lista_resultados.getDouble("cobertura_percentual"),
                lista_resultados.getBoolean("possui_atendimento_24h"),
                FormaPagamentoPreferencial.fromTexto(
                    lista_resultados.getString("forma_pagamento_preferencial")
                )
            );
        }

        lista_resultados.close();
        comando.close();

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        seguradora = null;
    }

    return seguradora;
}

```

```

public static String inserirSeguradora(Seguradora seguradora) {
    String sql = "INSERT INTO seguradoras (nome, cidade, cobertura_percentual,
    possui_atendimento_24h, forma_pagamento_preferencial) VALUES (?, ?, ?, ?, ?)";
}

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getNome());
    comando.setString(2, seguradora.getCidade());
    comando.setDouble(3, seguradora.getCoberturaPercentual());
    comando.setBoolean(4, seguradora.getPossuiAtendimento24h());
    comando.setString(5, seguradora.getFormaPagamentoPreferencial().toString());

    comando.executeUpdate();
    comando.close();

    return null;

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
    return "Erro na Inserção da Seguradora no BD";
}
}

```

```

public static String alterarSeguradora(Seguradora seguradora) {
    String sql = "UPDATE seguradoras SET cidade = ?, cobertura_percentual = ?,
possui_atendimento_24h = ?, forma_pagamento_preferencial = ?"
        + " WHERE nome = ?";

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getCidade());
    comando.setDouble(2, seguradora.getCoberturaPercentual());
    comando.setBoolean(3, seguradora.getPossuiAtendimento24h());
    comando.setString(4, seguradora.getFormaPagamentoPreferencial().toString());
    comando.setString(5, seguradora.getNome());

    comando.executeUpdate();
    comando.close();

```

```

        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração da Seguradora no BD";
    }
}

public static String removerSeguradora(String nome) {
    String sql = "DELETE FROM seguradoras WHERE nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);

        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção da Seguradora no BD";
    }
}
}

```

\$\$\$ src.entidades.Sinistro

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Sinistro {

    public enum GrauMonta {
        PEQUENA("pequena"),
        MEDIA("media"),
        GRANDE("grande");

        private final String texto;

        GrauMonta(String texto) {
            this.texto = texto;
        }

        public static GrauMonta fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Grau de monta não informado");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT)
                .replace("á", "a")
                .replace("à", "a")
                .replace("â", "a")
                .replace("ã", "a")
                .replace("é", "e")
                .replace("ê", "e")
                .replace("í", "i")
```

```
.replace("ó", "o")
.replace("ô", "o")
.replace("õ", "o")
.replace("ú", "u")
.replace("ç", "c");
```

```
if (normalizado.contains("pequena")) return PEQUENA;
if (normalizado.contains("media")) return MEDIA;
if (normalizado.contains("grande")) return GRANDE;
```

```
throw new IllegalArgumentException("Grau de monta inválido: " + texto);
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
private String segurado;
private String telefone;
private String cidade;
private GrauMonta grau_monta;
private boolean perda_total;
private ArrayList<Pecas> pecas;
```

```
public Sinistro(
    String segurado,
    String telefone,
    String cidade,
    GrauMonta grau_monta,
    boolean perda_total,
    ArrayList<Pecas> pecas
) {
    this.segurado = segurado;
    this.telefone = telefone;
    this.cidade = cidade;
```

```

        this.grau_monta = grau_monta != null ? grau_monta : GrauMonta.PEQUENA;
        this.perda_total = perda_total;
        this.pecas = pecas != null ? pecas : new ArrayList<Pecas>();
    }

    public Sinistro(
        String segurado,
        String telefone,
        String cidade,
        GrauMonta grau_monta,
        boolean perda_total
    ) {
        this(segurado, telefone, cidade, grau_monta, perda_total, new ArrayList<Pecas>());
    }

    public Sinistro(String segurado, String telefone, String cidade) {
        this(segurado, telefone, cidade, GrauMonta.PEQUENA, false);
    }

    public Sinistro(String segurado, String telefone) {
        this(segurado, telefone, null, GrauMonta.PEQUENA, false);
    }

    public String getSegurado() {
        return segurado;
    }

    public String getTelefone() {
        return telefone;
    }

    public String getCidade() {
        return cidade;
    }

    public GrauMonta getGrauMonta() {
        return grau_monta;
    }

```

```
}
```

```
public boolean getPerdaTotal() {  
    return perda_total;  
}
```

```
public boolean isPerdaTotal() {  
    return perda_total;  
}
```

```
public Pecas[] getPecas() {  
    return pecas.toArray(new Pecas[pecas.size()]);  
}
```

```
public void setSegurado(String segurado) {  
    this.segurado = segurado;  
}
```

```
public void setTelefone(String telefone) {  
    this.telefone = telefone;  
}
```

```
public void setCidade(String cidade) {  
    this.cidade = cidade;  
}
```

```
public void setGrauMonta(GrauMonta grau_monta) {  
    this.grau_monta = grau_monta != null ? grau_monta : GrauMonta.PEQUENA;  
}
```

```
public void setPerdaTotal(boolean perda_total) {  
    this.perda_total = perda_total;  
}
```

```
public void setPecas(Pecas[] pecas) {  
    this.pecas = new ArrayList<Pecas>();  
    if (pecas != null) {
```



```

        for (Pecas peca : pecas) {
            if (peca != null) {
                this.pecas.add(peca);
            }
        }
    }
}

```

```

public void adicionarPeca(Pecas peca) {
    if (peca != null) {
        if (pecas == null) {
            pecas = new ArrayList<Pecas>();
        }
        pecas.add(peca);
    }
}

```

@Override

```

public String toString() {
    String cidade_texto = cidade != null ? cidade : "";
    return segurado + " - " + cidade_texto + " (" + grau_monta + ")";
}

```

```

public Sinistro getVisao() {
    return new Sinistro(
        segurado,
        telefone,
        cidade,
        grau_monta,
        perda_total,
        pecas
    );
}

```

```

private static Sinistro criarVisao(ResultSet resultado) throws SQLException {
    String segurado = resultado.getString("segurado");
    String telefone = resultado.getString("telefone");
}

```

```

String cidade = resultado.getString("cidade");
GrauMonta grau_monta = GrauMonta.fromTexto(resultado.getString("grau_monta"));
boolean perda_total = resultado.getBoolean("perda_total");
return new Sinistro(segurado, telefone, cidade, grau_monta, perda_total);
}

```

```

private static Sinistro criarVisaoLegado(ResultSet resultado) throws SQLException {
    String segurado = resultado.getString("nome");
    String telefone = resultado.getString("telefone");
    String cidade = null;
    GrauMonta grau_monta = GrauMonta.fromTexto(resultado.getString("grau_monta"));
    boolean perda_total = resultado.getBoolean("perda_total");
    return new Sinistro(segurado, telefone, cidade, grau_monta, perda_total);
}

```

```

public static Sinistro[] getVisoes() {
    String sql = "SELECT segurado, telefone, cidade, grau_monta, perda_total FROM sinistros";
    ResultSet lista_resultados = null;
    ArrayList<Sinistro> visoes = new ArrayList<>();

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {
                visoes.add(criarVisao(lista_resultados));
            } catch (IllegalArgumentException execucao_enum) {
                // Ignora registros com valor fora do domínio esperado.
            }
        }

        lista_resultados.close();
        comando.close();
    } catch (SQLException execucao_sql) {
        try {
            String sql_legacy = "SELECT numero, nome, telefone, grau_monta, perda_total FROM

```

```

sinistros";

PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
lista_resultados = comando_legado.executeQuery();

while (lista_resultados.next()) {
    try {
        visoes.add(criarVisaoLegado(lista_resultados));
    } catch (IllegalArgumentException excecao_enum) {
        // Ignora registros inválidos.
    }
}

lista_resultados.close();
comando_legado.close();
} catch (SQLException excecao_legado) {
    excecao_legado.printStackTrace();
}
}

return visoes.toArray(new Sinistro[visoes.size()]);
}

public static Sinistro buscarSinistro(String segurado) {
    String sql = "SELECT segurado, telefone, cidade, grau_monta, perda_total FROM sinistros
WHERE segurado = ?";

    ResultSet lista_resultados = null;
    Sinistro sinistro = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, segurado);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {
                sinistro = criarVisao(lista_resultados);
                sinistro.setPecas(Pecas.buscarPecasPorSinistro(segurado));
            }
        }
    }
}

```

```

    } catch (IllegalArgumentException excecao_enum) {
        sinistro = null;
    }
}

lista_resultados.close();
comando.close();
} catch (SQLException excecao_sql) {
    try {
        String sql_legado = "SELECT numero, nome, telefone, grau_monta, perda_total FROM
sinistros WHERE nome = ?";
        PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
        comando_legado.setString(1, segurado);
        lista_resultados = comando_legado.executeQuery();

        while (lista_resultados.next()) {
            try {
                sinistro = criarVisaoLegado(lista_resultados);
                sinistro.setPecas(new Pecas[0]);
            } catch (IllegalArgumentException excecao_enum) {
                sinistro = null;
            }
        }

        lista_resultados.close();
        comando_legado.close();
    } catch (SQLException excecao_legado) {
        excecao_legado.printStackTrace();
        sinistro = null;
    }
}

return sinistro;
}

public static String inserirSinistro(Sinistro sinistro) {
    String sql = "INSERT INTO sinistros (segurado, telefone, cidade, grau_monta, perda_total)

```

VALUES (?, ?, ?, ?, ?)";

```
try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    comando.setString(1, sinistro.getSegurado());
    comando.setString(2, sinistro.getTelefone());
    comando.setString(3, sinistro.getCidade());
    comando.setString(4, sinistro.getGrauMonta().toString());
    comando.setBoolean(5, sinistro.getPerdaTotal());
    comando.executeUpdate();
    comando.close();
    return null;
} catch (SQLException excecao_sql) {
    return inserirSinistroLegado(sinistro);
}
}
```

```
public static String alterarSinistro(Sinistro sinistro) {
    String sql = "UPDATE sinistros SET telefone = ?, cidade = ?, grau_monta = ?, perda_total = ?
    WHERE segurado = ?";
```

```
try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    comando.setString(1, sinistro.getTelefone());
    comando.setString(2, sinistro.getCidade());
    comando.setString(3, sinistro.getGrauMonta().toString());
    comando.setBoolean(4, sinistro.getPerdaTotal());
    comando.setString(5, sinistro.getSegurado());
    comando.executeUpdate();
    comando.close();
    return null;
} catch (SQLException excecao_sql) {
    return alterarSinistroLegado(sinistro);
}
}
```

```
public static String removerSinistro(String segurado) {
```

```
String sql = "DELETE FROM sinistros WHERE segurado = ?";
```

```
try {  
    PreparedStatement comando = BD.conexao.prepareStatement(sql);  
    comando.setString(1, segurado);  
    comando.executeUpdate();  
    comando.close();  
    return null;  
} catch (SQLException excecao_sql) {  
    return removerSinistroLegado(segurado);  
}  
}
```

```
public static boolean existeSinistroMesmosAtributos(Sinistro sinistro) {  
    String sql = "SELECT COUNT(segurado) FROM sinistros WHERE segurado = ? AND  
telefone = ? AND cidade = ? AND grau_monta = ? AND perda_total = ?";  
    ResultSet lista_resultados = null;  
    int n_sinistros_mesmos_atributos = 0;  
  
    try {  
        PreparedStatement comando = BD.conexao.prepareStatement(sql);  
        comando.setString(1, sinistro.getSegurado());  
        comando.setString(2, sinistro.getTelefone());  
        comando.setString(3, sinistro.getCidade());  
        comando.setString(4, sinistro.getGrauMonta().toString());  
        comando.setBoolean(5, sinistro.getPerdaTotal());  
        lista_resultados = comando.executeQuery();  
  
        while (lista_resultados.next()) {  
            n_sinistros_mesmos_atributos = lista_resultados.getInt(1);  
        }  
  
        lista_resultados.close();  
        comando.close();  
    } catch (SQLException excecao_sql) {  
        try {  
            String sql_legado = "SELECT COUNT(numero) FROM sinistros WHERE nome = ?
```

```

AND telefone = ? AND grau_monta = ? AND perda_total = ?";

PreparedStatement comando_legado = BD.conexao.prepareStatement(sql_legado);
comando_legado.setString(1, sinistro.getSegurado());
comando_legado.setString(2, sinistro.getTelefone());
comando_legado.setString(3, sinistro.getGrauMonta().toString());
comando_legado.setBoolean(4, sinistro.getPerdaTotal());
lista_resultados = comando_legado.executeQuery();

while (lista_resultados.next()) {
    n_sinistros_mesmos_atributos = lista_resultados.getInt(1);
}

lista_resultados.close();
comando_legado.close();
} catch (SQLException excecao_legado) {
    excecao_legado.printStackTrace();
}
}

return n_sinistros_mesmos_atributos > 0;
}

```

```

private static int ultimoNumeroLegado() {
    String sql = "SELECT MAX(numero) FROM sinistros";
    ResultSet lista_resultados = null;
    int numero = 0;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            numero = lista_resultados.getInt(1);
        }

        lista_resultados.close();
        comando.close();
    }
}

```

```
} catch (SQLException excecao_sql) {  
    excecao_sql.printStackTrace();  
}
```

```
return numero;  
}
```

```
private static String inserirSinistroLegado(Sinistro sinistro) {  
    String sql = "INSERT INTO sinistros (numero, nome, telefone, grau_monta, perda_total)  
VALUES (?, ?, ?, ?, ?)";
```

```
try {  
    PreparedStatement comando = BD.conexao.prepareStatement(sql);  
    comando.setInt(1, ultimoNumeroLegado() + 1);  
    comando.setString(2, sinistro.getSegurado());  
    comando.setString(3, sinistro.getTelefone());  
    comando.setString(4, sinistro.getGrauMonta().toString());  
    comando.setBoolean(5, sinistro.getPerdaTotal());  
    comando.executeUpdate();  
    comando.close();  
    return null;  
} catch (SQLException excecao_sql) {  
    excecao_sql.printStackTrace();  
    return "Erro na Inserção do Sinistro no BD";  
}  
}
```

```
private static String alterarSinistroLegado(Sinistro sinistro) {  
    String sql = "UPDATE sinistros SET telefone = ?, grau_monta = ?, perda_total = ? WHERE  
nome = ?";
```

```
try {  
    PreparedStatement comando = BD.conexao.prepareStatement(sql);  
    comando.setString(1, sinistro.getTelefone());  
    comando.setString(2, sinistro.getGrauMonta().toString());  
    comando.setBoolean(3, sinistro.getPerdaTotal());  
    comando.setString(4, sinistro.getSegurado());
```



```

        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração do Sinistro no BD";
    }
}

```

```

private static String removerSinistroLegado(String segurado) {
    String sql = "DELETE FROM sinistros WHERE nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, segurado);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção do Sinistro no BD";
    }
}
}

```

\$\$\$ src.interfaces.JanelaCadastroPecas

```
package interfaces;

import controles.ControladorCadastroSeguradoras;
import controles.ControladorCadastroPecas;
import controles.ControladorCadastroSinistros;
import javax.swing.JOptionPane;
import persistência.BD;

public class JanelaSistema extends javax.swing.JFrame {

    public JanelaSistema() {
        BD.criaConexao();
        initComponents();
    }

    @SuppressWarnings("unchecked")

    private void informarServiçoIndisponível() {
        JOptionPane.showMessageDialog(
            this,
            "Serviço Indisponível",
            "Informação",
            JOptionPane.INFORMATION_MESSAGE
        );
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
        sinistroMenu = new javax.swing.JMenu();
        cadastrar_sinistroMenuItem = new javax.swing.JMenuItem();
        seguradoraMenu = new javax.swing.JMenu();
        cadastrar_seguradoraItemMenu = new javax.swing.JMenuItem();
        peçaMenu = new javax.swing.JMenu();
```

```

cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
orçamentoMenu = new javax.swing.JMenu();
cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastro de Seguradoras e Peças");
setAlwaysOnTop(true);
addWindowListener(new java.awt.event.WindowAdapter() {
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

sinistroMenu.setText("Sinistro");

cadastrar_sinistroMenuItem.setText("Cadastrar");
cadastrar_sinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSinistro(evt);
    }
});
sinistroMenu.add(cadastrar_sinistroMenuItem);

seguradoras_orçamentoMenuBar.add(sinistroMenu);

seguradoraMenu.setText("Seguradora");

cadastrar_seguradoraItemMenu.setText("Cadastrar");
cadastrar_seguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrar_seguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);

```

```
peçaMenu.setText("Peça");
```

```
cadastrar_peçaMenuItem.setText("Cadastrar");
```

```
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarPeça(evt);  
    }  
});
```

```
peçaMenu.add(cadastrar_peçaMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(peçaMenu);
```

```
orçamentoMenu.setText("Orçamento");
```

```
cadastrar_orçamentoMenuItem.setText("Cadastrar");
```

```
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(cadastrar_orçamentoMenuItem);
```

```
pesquisar_orçamentoMenuItem.setText("Pesquisar");
```

```
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        pesquisarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(pesquisar_orçamentoMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(orçamentoMenu);
```

```
setJMenuBar(seguradoras_orçamentoMenuBar);
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);
```

```
layout.setHorizontalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
    .addGap(0, 400, Short.MAX_VALUE)  
);  
layout.setVerticalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
    .addGap(0, 278, Short.MAX_VALUE)  
);  
  
pack();  
} // </editor-fold>
```

```
private void cadastrarSeguradora(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSeguradoras(this);  
}
```

```
private void terminarSistema(java.awt.event.WindowEvent evt) {  
    BD.fechaConexao();  
    System.exit(0);  
}
```

```
private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarPeça(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroPecas(this);  
}
```

```
private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarSinistro(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSinistros();  
}
```

```

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new JanelaSistema().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;
private javax.swing.JMenuItem cadastrar_peçaMenuItem;
private javax.swing.JMenuItem cadastrar_seguradoraItemMenu;
private javax.swing.JMenuItem cadastrar_sinistroMenuItem;
private javax.swing.JMenu orçamentoMenu;
private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;
private javax.swing.JMenu peçaMenu;
private javax.swing.JMenu seguradoraMenu;
private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;
private javax.swing.JMenu sinistroMenu;
// End of variables declaration

}

```

\$\$\$ src.interfaces.JanelaCadastroSeguradoras

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroSeguradoras;
import entidades.Seguradora;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroSeguradoras extends javax.swing.JFrame {

    ControladorCadastroSeguradoras controlador;
    Seguradora[] seguradoras_cadastradas;

    public JanelaCadastroSeguradoras(ControladorCadastroSeguradoras controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSeguradoras(
        ControladorCadastroSeguradoras controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        seguradoras_cadastradas = Seguradora.getVisoes();
        initComponents();
        atualizarSeguradorasCadastradas();
        setSize(new java.awt.Dimension(760, 540));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
```

```
        setLocationByPlatform(true);  
        return;  
    }
```

```
    posicionarRelativaAoOwner(owner);  
}
```

```
private void posicionarRelativaAoOwner(Frame owner) {  
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;  
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;  
    setLocation(x, y);  
}
```

```
private Seguradora localizarSeguradora(String nome) {  
    if (nome == null) return null;  
    for (Seguradora visao : seguradoras_cadastradas) {  
        if (nome.equals(visao.getNome())) return visao;  
    }  
    return null;  
}
```

```
private void atualizarSeguradorasCadastradas() {  
    atualizarSeguradorasCadastradas(null);  
}
```

```
private void atualizarSeguradorasCadastradas(String nome_selecionado) {  
    seguradoras_cadastradas = Seguradora.getVisoes();  
    seguradoras_cadastradasComboBox.setModel(  
        new DefaultComboBoxModel(seguradoras_cadastradas)  
    );  
}
```

```
    Seguradora visao_selecionada = localizarSeguradora(nome_selecionado);  
    if (visao_selecionada != null) {  
        seguradoras_cadastradasComboBox.setSelectedItem(visao_selecionada);  
    }  
}
```



```
private void informarErro(String mensagem) {  
    JOptionPane.showMessageDialog(  
        this,  
        mensagem,  
        "Erro",  
        JOptionPane.ERROR_MESSAGE  
    );  
}
```

```
private Seguradora.FormaPagamentoPreferencial  
obterFormaPagamentoPreferencialSelecionada() {  
    Object selecionado = formaPagamentoPreferencialComboBox.getSelectedItem();  
    if (selecionado == null) {  
        return Seguradora.FormaPagamentoPreferencial.BOLETO;  
    }  
  
    return Seguradora.FormaPagamentoPreferencial.fromTexto(selecionado.toString());  
}
```

```
private Seguradora obterSeguradoraInformada() {  
  
    String nome = nomeTextField.getText();  
    if (nome.isEmpty()) return null;  
  
    String cidade = cidadeTextField.getText();  
    if (cidade.isEmpty()) cidade = null;  
  
    String cobertura_str = coberturaPercentualTextField.getText();  
    if (cobertura_str.isEmpty()) return null;  
  
    double cobertura_percentual;  
  
    try {  
        cobertura_percentual =  
            Double.parseDouble(cobertura_str.replace(",", "."));  
    } catch (NumberFormatException e) {  
        return null;  
    }
```

```
}
```

```
boolean possui_atendimento_24h =  
    possuiAtendimento24hCheckBox.isSelected();
```

```
return new Seguradora(  
    nome,  
    cidade,  
    cobertura_percentual,  
    possui_atendimento_24h,  
    obterFormaPagamentoPreferencialSelecionada()  
);  
}
```

```
private Seguradora getVisaoAlterada(String nome) {  
    for (Seguradora visao : seguradoras_cadastradas) {  
        if (visao.getNome().equals(nome)) return visao;  
    }  
    return null;  
}
```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
```

```
private void initComponents() {  
    java.awt.GridBagConstraints gridBagConstraints;  
  
    comandosPanel = new javax.swing.JPanel();  
    inserirSeguradora = new javax.swing.JButton();  
    alterarSeguradora = new javax.swing.JButton();  
    consultarSeguradora = new javax.swing.JButton();  
    removerSeguradora = new javax.swing.JButton();  
    limparCampos = new javax.swing.JButton();  
    nomeLabel = new javax.swing.JLabel();  
    nomeTextField = new javax.swing.JTextField();  
    cidadeLabel = new javax.swing.JLabel();  
    cidadeTextField = new javax.swing.JTextField();  
    coberturaPercentualLabel = new javax.swing.JLabel();
```

```
coberturaPercentualTextField = new javax.swing.JTextField();
seguradorasCadastradasLabel = new javax.swing.JLabel();
seguradoras_cadastradasComboBox = new javax.swing.JComboBox();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Seguradoras");
setMinimumSize(new java.awt.Dimension(620, 360));
setPreferredSize(new java.awt.Dimension(700, 460));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSeguradora.setText("Inserir");
inserirSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSeguradora(evt);
    }
});
comandosPanel.add(inserirSeguradora);

alterarSeguradora.setText("Alterar");
alterarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSeguradora(evt);
    }
});
comandosPanel.add(alterarSeguradora);

consultarSeguradora.setText("Consultar");
consultarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSeguradora(evt);
    }
});
comandosPanel.add(consultarSeguradora);

removerSeguradora.setText("Remover");
removerSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        removerSeguradora(evt);
    }
});
comandosPanel.add(removerSeguradora);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
nomeLabel.setMaximumSize(null);
nomeLabel.setMinimumSize(null);
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
nomeTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        nomeTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 80;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
cidadeLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(cidadeLabel, gridBagConstraints);
```

```
cidadeTextField.setColumns(50);
cidadeTextField.setPreferredSize(new java.awt.Dimension(180, 20));
cidadeTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cidadeTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 20;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(cidadeTextField, gridBagConstraints);
```

```
coberturaPercentualLabel.setText("Cobertura Percentual");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(coberturaPercentualLabel, gridBagConstraints);
```

```

coberturaPercentualTextField.setColumns(50);
coberturaPercentualTextField.setPreferredSize(new java.awt.Dimension(100, 20));
coberturaPercentualTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        coberturaPercentualTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(coberturaPercentualTextField, gridBagConstraints);

seguradorasCadastradasLabel.setText("Seguradoras Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(0, 12, 0, 5);
getContentPane().add(seguradorasCadastradasLabel, gridBagConstraints);

seguradoras_cadastradasComboBox.setModel(new
DefaultComboBoxModel(seguradoras_cadastradas));
seguradoras_cadastradasComboBox.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        seguradoras_cadastradasComboBoxActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoras_cadastradasComboBox, gridBagConstraints);

pack();
} // </editor-fold>

```

```
private void nomeTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void cidadeTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void coberturaPercentualTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void alterarSeguradora(java.awt.event.ActionEvent evt) {  
    Seguradora seguradora =  
        obterSeguradoraInformada();
```

```
    String mensagem_erro = null;
```

```
    if (seguradora != null)  
        mensagem_erro =  
            controlador.alterarSeguradora(seguradora);
```

```
    else  
        mensagem_erro =  
            "Algum atributo da Seguradora não foi informado.";
```

```
    if (mensagem_erro == null) {
```

```
        Seguradora visao =  
            getVisaoAlterada(seguradora.getNome());
```

```
        if (visao != null) {
```

```
            visao.setCidade(  
                seguradora.getCidade()  
            );
```

```

        visao.setCoberturaPercentual(
            seguradora.getCoberturaPercentual()
        );

        visao.setPossuiAtendimento24h(
            seguradora.getPossuiAtendimento24h()
        );

        visao.setFormaPagamentoPreferencial(
            seguradora.getFormaPagamentoPreferencial()
        );

        seguradoras_cadastradasComboBox.updateUI();

        seguradoras_cadastradasComboBox
            .setSelectedItem(visao);
    }

} else {
    informarErro(mensagem_erro);
}
}

private void inserirSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora seguradora =
        obterSeguradoraInformada();

    String mensagem_erro = null;

    if (seguradora != null)
        mensagem_erro =
            controlador.inserirSeguradora(seguradora);
    else
        mensagem_erro =
            "Algum atributo da Seguradora não foi informado";

    if (mensagem_erro == null) {

```



```

        Seguradora visao = seguradora.getVisao();
        atualizarSeguradorasCadastradas(visao.getNome());

    } else {
        informarErro(mensagem_erro);
    }
}

private void consultarSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

    Seguradora seguradora = null;

    String mensagem_erro = null;

    if (visao != null) {

        seguradora =
            Seguradora.buscarSeguradora(
                visao.getNome()
            );

        if (seguradora == null)
            mensagem_erro =
                "Seguradora não cadastrada";

    } else {
        mensagem_erro =
            "Nenhuma Seguradora selecionada";
    }

    if (mensagem_erro == null) {

```

```

nomeTextField.setText(
    seguradora.getNome()
);

String cidade =
    seguradora.getCidade();

if (cidade == null)
    cidade = "";

cidadeTextField.setText(cidade);

coberturaPercentualTextField.setText(
    String.valueOf(
        seguradora
            .getCoberturaPercentual()
    )
);

possuiAtendimento24hCheckBox.setSelected(
    seguradora.getPossuiAtendimento24h()
);

formaPagamentoPreferencialComboBox.setSelectedItem(
    seguradora.getFormaPagamentoPreferencial().toString()
);

} else {
    informarErro(mensagem_erro);
}
}

private void removerSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

```

```
String mensagem_erro = null;
```

```
if (visao != null)
```

```
    mensagem_erro =
```

```
        controlador.removerSeguradora(
```

```
            visao.getNome()
```

```
        );
```

```
else
```

```
    mensagem_erro =
```

```
        "Nenhuma Seguradora selecionada";
```

```
if (mensagem_erro == null) {
```

```
    atualizarSeguradorasCadastradas();
```

```
    limparCampos(evt);
```

```
} else {
```

```
    informarErro(mensagem_erro);
```

```
}
```

```
}
```

```
private void limparCampos(java.awt.event.ActionEvent evt) {
```

```
    nomeTextField.setText("");
```

```
    cidadeTextField.setText("");
```

```
    coberturaPercentualTextField.setText("");
```

```
    possuiAtendimento24hCheckBox.setSelected(false);
```

```
    formaPagamentoPreferencialComboBox.setSelectedItem("boleto");
```

```
}
```

```
private void seguradoras_cadastradasComboBoxActionPerformed(java.awt.event.ActionEvent  
evt) {
```

```
}
```

```
// Variables declaration - do not modify
private javax.swing.JButton alterarSeguradora;
private javax.swing.JLabel cidadeLabel;
private javax.swing.JTextField cidadeTextField;
private javax.swing.JLabel coberturaPercentualLabel;
private javax.swing.JTextField coberturaPercentualTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarSeguradora;
private javax.swing.JButton inserirSeguradora;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JButton removerSeguradora;
private javax.swing.JLabel seguradorasCadastradasLabel;
private javax.swing.JComboBox seguradoras_cadastradasComboBox;
// End of variables declaration
}
```

\$\$\$ src.interfaces.JanelaCadastroSinistros

```
package interfaces;

import controles.ControladorCadastroSinistros;
import entidades.Sinistro;
import java.awt.Frame;
import java.util.Objects;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JOptionPane;

public class JanelaCadastroSinistros extends javax.swing.JFrame {

    ControladorCadastroSinistros controlador;
    Sinistro[] sinistros_cadastrados;

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador, Frame owner) {
        this.controlador = controlador;
        sinistros_cadastrados = Sinistro.getVisoes();
        initComponents();
        atualizarSinistrosCadastrados();
        setSize(new java.awt.Dimension(720, 470));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
            setLocationByPlatform(true);
            return;
        }
    }
}
```

```
}
```

```
int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;  
int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;  
setLocation(x, y);
```

```
}
```

```
private Sinistro localizarSinistro(String segurado) {  
    if (sinistros_cadastrados == null) return null;  
    for (Sinistro visao : sinistros_cadastrados) {  
        if (Objects.equals(visao.getSegurado(), segurado)) return visao;  
    }  
    return null;  
}
```

```
private void atualizarSinistrosCadastrados() {  
    atualizarSinistrosCadastrados(null);  
}
```

```
private void atualizarSinistrosCadastrados(String segurado_selecionado) {  
    sinistros_cadastrados = Sinistro.getVisoes();  
    sinistros_cadastradosComboBox.setModel(new  
DefaultComboBoxModel(sinistros_cadastrados));
```

```
    Sinistro visao_selecionada = localizarSinistro(segurado_selecionado);  
    if (visao_selecionada != null) {  
        sinistros_cadastradosComboBox.setSelectedItem(visao_selecionada);  
    }  
}
```

```
private void informarErro(String mensagem) {  
    JOptionPane.showMessageDialog(this, mensagem, "Erro",  
JOptionPane.ERROR_MESSAGE);  
}
```

```
private Sinistro obterSinistroInformado() {  
    String segurado = numeroTextField.getText().trim();
```

```

if (segurado.isEmpty()) return null;

String telefone = telefoneTextField.getText().trim();
if (telefone.isEmpty()) return null;

String cidade = clienteTextField.getText().trim();
if (cidade.isEmpty()) cidade = null;

Sinistro.GrauMonta grau_monta =
    (Sinistro.GrauMonta) grauMontaComboBox.getSelectedItem();
if (grau_monta == null) return null;

boolean perda_total = perdaTotalCheckBox.isSelected();

return new Sinistro(segurado, telefone, cidade, grau_monta, perda_total);
}

private Sinistro getVisaoAlterada(String asegurado) {
    for (Sinistro visao : sinistros_cadastrados) {
        if (Objects.equals(visao.getSegurado(), asegurado)) return visao;
    }
    return null;
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSinistro = new javax.swing.JButton();
    alterarSinistro = new javax.swing.JButton();
    consultarSinistro = new javax.swing.JButton();
    removerSinistro = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    sinistrosCadastradosLabel = new javax.swing.JLabel();
    sinistros_cadastradosComboBox = new javax.swing.JComboBox();

```

```
numeroLabel = new javax.swing.JLabel();
numeroTextField = new javax.swing.JTextField();
clienteLabel = new javax.swing.JLabel();
clienteTextField = new javax.swing.JTextField();
telefoneLabel = new javax.swing.JLabel();
telefoneTextField = new javax.swing.JTextField();
grauMontaLabel = new javax.swing.JLabel();
grauMontaComboBox = new javax.swing.JComboBox();
perdaTotalLabel = new javax.swing.JLabel();
perdaTotalCheckBox = new javax.swing.JCheckBox();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Sinistros");
setMinimumSize(new java.awt.Dimension(680, 420));
setPreferredSize(new java.awt.Dimension(720, 470));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSinistro.setText("Inserir");
inserirSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSinistro(evt);
    }
});
comandosPanel.add(inserirSinistro);

alterarSinistro.setText("Alterar");
alterarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSinistro(evt);
    }
});
comandosPanel.add(alterarSinistro);

consultarSinistro.setText("Consultar");
consultarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSinistro(evt);
    }
});
```



```
    }  
});  
comandosPanel.add(consultarSinistro);
```

```
removerSinistro.setText("Remover");  
removerSinistro.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        removerSinistro(evt);  
    }  
});  
comandosPanel.add(removerSinistro);
```

```
limparCampos.setText("Limpar");  
limparCampos.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        limparCampos(evt);  
    }  
});  
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 6;  
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);  
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
sinistrosCadastradosLabel.setText("Sinistros Cadastrados");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 0;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(sinistrosCadastradosLabel, gridBagConstraints);
```

```
sinistros_cadastradosComboBox.setModel(new  
DefaultComboBoxModel(sinistros_cadastrados));  
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(sinistros_cadastradosComboBox, gridBagConstraints);
```

```
numeroLabel.setText("Segurado");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(numeroLabel, gridBagConstraints);
```

```
numeroTextField.setColumns(10);
numeroTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(numeroTextField, gridBagConstraints);
```

```
clienteLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteLabel, gridBagConstraints);
```

```
clienteTextField.setColumns(24);
clienteTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(clienteTextField, gridBagConstraints);
```

```
telefoneLabel.setText("Telefone");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 3;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(telefoneLabel, gridBagConstraints);
```

```
telefoneTextField.setColumns(16);  
telefoneTextField.setPreferredSize(new java.awt.Dimension(160, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 3;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(telefoneTextField, gridBagConstraints);
```

```
grauMontaLabel.setText("Grau de Monta");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 4;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(grauMontaLabel, gridBagConstraints);
```

```
grauMontaComboBox.setModel(new  
DefaultComboBoxModel(Sinistro.GrauMonta.values()));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 4;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(grauMontaComboBox, gridBagConstraints);
```

```

perdaTotalLabel.setText("Perda Total");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(perdaTotalLabel, gridBagConstraints);

perdaTotalCheckBox.setText("Sim");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(perdaTotalCheckBox, gridBagConstraints);

pack();
} // </editor-fold>

private void limparCampos(java.awt.event.ActionEvent evt) {
    numeroTextField.setText("");
    numeroTextField.setEditable(true);
    clienteTextField.setText("");
    telefoneTextField.setText("");
    grauMontaComboBox.setSelectedIndex(0);
    perdaTotalCheckBox.setSelected(false);
    sinistros_cadastradosComboBox.setSelectedIndex(-1);
}

private void consultarSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;
    if (visao == null) {
        mensagem_erro = "Nenhum Sinistro selecionado";
    } else {
        Sinistro sinistro = Sinistro.buscarSinistro(visao.getSegurado());
        if (sinistro == null) {

```

```

        mensagem_erro = "Sinistro não cadastrado";
    } else {
        numeroTextField.setText(sinistro.getSegurado());
        numeroTextField.setEditable(false);
        clienteTextField.setText(sinistro.getCidade());
        telefoneTextField.setText(sinistro.getTelefone());
        grauMontaComboBox.setSelectedItem(sinistro.getGrauMonta());
        perdaTotalCheckBox.setSelected(sinistro.getPerdaTotal());
    }
}

if (mensagem_erro != null) informarErro(mensagem_erro);
}

private void inserirSinistro(java.awt.event.ActionEvent evt) {
    Sinistro sinistro = obterSinistroInformado();
    String mensagem_erro = null;

    if (sinistro != null) {
        mensagem_erro = controlador.inserirSinistro(sinistro);
    } else {
        mensagem_erro = "Algum atributo do Sinistro não foi informado";
    }

    if (mensagem_erro == null) {
        atualizarSinistrosCadastrados(sinistro.getSegurado());
        numeroTextField.setText(sinistro.getSegurado());
        numeroTextField.setEditable(false);
        sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getSegurado()));
    } else {
        informarErro(mensagem_erro);
    }
}

private void alterarSinistro(java.awt.event.ActionEvent evt) {
    Sinistro sinistro = obterSinistroInformado();
    String mensagem_erro = null;

```

```

if (sinistro != null) {
    mensagem_erro = controlador.alterarSinistro(sinistro);
} else {
    mensagem_erro = "Algum atributo do Sinistro não foi informado";
}

if (mensagem_erro == null) {
    atualizarSinistrosCadastrados(sinistro.getSegurado());
    sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getSegurado()));
} else {
    informarErro(mensagem_erro);
}
}

private void removerSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;

    if (visao != null) {
        mensagem_erro = controlador.removerSinistro(visao.getSegurado());
    } else {
        mensagem_erro = "Nenhum Sinistro selecionado";
    }

    if (mensagem_erro == null) {
        atualizarSinistrosCadastrados();
        limparCampos(evt);
    } else {
        informarErro(mensagem_erro);
    }
}

// Variables declaration - do not modify
private javax.swing.JButton alterarSinistro;
private javax.swing.JButton consultarSinistro;
private javax.swing.JTextField clienteTextField;

```

```
private javax.swing.JLabel clienteLabel;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton inserirSinistro;
private javax.swing.JButton limparCampos;
private javax.swing.JComboBox grauMontaComboBox;
private javax.swing.JLabel grauMontaLabel;
private javax.swing.JLabel numeroLabel;
private javax.swing.JTextField numeroTextField;
private javax.swing.JCheckBox perdaTotalCheckBox;
private javax.swing.JLabel perdaTotalLabel;
private javax.swing.JButton removerSinistro;
private javax.swing.JLabel sinistrosCadastradosLabel;
private javax.swing.JComboBox sinistros_cadastradosComboBox;
private javax.swing.JLabel telefoneLabel;
private javax.swing.JTextField telefoneTextField;
// End of variables declaration
}
```

\$\$\$ src.interfaces.JanelaSistema

```
package interfaces;

import controles.ControladorCadastroSeguradoras;
import controles.ControladorCadastroPecas;
import controles.ControladorCadastroSinistros;
import javax.swing.JOptionPane;
import persistência.BD;

public class JanelaSistema extends javax.swing.JFrame {

    public JanelaSistema() {
        BD.criaConexao();
        initComponents();
    }

    @SuppressWarnings("unchecked")

    private void informarServiçoIndisponível() {
        JOptionPane.showMessageDialog(
            this,
            "Serviço Indisponível",
            "Informação",
            JOptionPane.INFORMATION_MESSAGE
        );
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
        sinistroMenu = new javax.swing.JMenu();
        cadastrar_sinistroMenuItem = new javax.swing.JMenuItem();
        seguradoraMenu = new javax.swing.JMenu();
        cadastrar_seguradoraItemMenu = new javax.swing.JMenuItem();
        peçaMenu = new javax.swing.JMenu();
```



```

cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
orçamentoMenu = new javax.swing.JMenu();
cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastro de Seguradoras e Peças");
setAlwaysOnTop(true);
addWindowListener(new java.awt.event.WindowAdapter() {
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

sinistroMenu.setText("Sinistro");

cadastrar_sinistroMenuItem.setText("Cadastrar");
cadastrar_sinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSinistro(evt);
    }
});
sinistroMenu.add(cadastrar_sinistroMenuItem);

seguradoras_orçamentoMenuBar.add(sinistroMenu);

seguradoraMenu.setText("Seguradora");

cadastrar_seguradoraItemMenu.setText("Cadastrar");
cadastrar_seguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrar_seguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);

```

```
peçaMenu.setText("Peça");
```

```
cadastrar_peçaMenuItem.setText("Cadastrar");
```

```
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarPeça(evt);  
    }  
});
```

```
peçaMenu.add(cadastrar_peçaMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(peçaMenu);
```

```
orçamentoMenu.setText("Orçamento");
```

```
cadastrar_orçamentoMenuItem.setText("Cadastrar");
```

```
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(cadastrar_orçamentoMenuItem);
```

```
pesquisar_orçamentoMenuItem.setText("Pesquisar");
```

```
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        pesquisarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(pesquisar_orçamentoMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(orçamentoMenu);
```

```
setJMenuBar(seguradoras_orçamentoMenuBar);
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);
```

```
layout.setHorizontalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGap(0, 400, Short.MAX_VALUE)  
);  
layout.setVerticalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGap(0, 278, Short.MAX_VALUE)  
);  
  
pack();  
} // </editor-fold>
```

```
private void cadastrarSeguradora(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSeguradoras(this);  
}
```

```
private void terminarSistema(java.awt.event.WindowEvent evt) {  
    BD.fechaConexao();  
    System.exit(0);  
}
```

```
private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarPeça(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroPecas(this);  
}
```

```
private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarSinistro(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSinistros();  
}
```

```
public static void main(String args[]) {  
    java.awt.EventQueue.invokeLater(new Runnable() {  
        public void run() {  
            new JanelaSistema().setVisible(true);  
        }  
    });  
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;  
private javax.swing.JMenuItem cadastrar_peçaMenuItem;  
private javax.swing.JMenuItem cadastrar_seguradoraItemMenu;  
private javax.swing.JMenuItem cadastrar_sinistroMenuItem;  
private javax.swing.JMenu orçamentoMenu;  
private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;  
private javax.swing.JMenu peçaMenu;  
private javax.swing.JMenu seguradoraMenu;  
private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;  
private javax.swing.JMenu sinistroMenu;
```

```
// End of variables declaration
```

```
}
```

\$\$\$ src.persistência.BD

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class BD {

    static final String URL_BD = "jdbc:mysql://localhost/banco";
    static final String USUARIO = "root";
    static final String SENHA = "admin";
    public static Connection conexao = null;

    public static void criaConexao() {
        try {
            conexao = DriverManager.getConnection(URL_BD, USUARIO, SENHA);
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }

    public static void fechaConexao() {
        try {
            if (conexao != null) {
                conexao.close();
            }
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }
}
```